

AXIOM

Autonomous Data Scientist Platform
Complete Frontend & Backend Reference

A production-grade multi-agent AI system that performs end-to-end machine learning workflows autonomously — from raw CSV ingestion to trained models, SHAP explainability, and publishable reports.

CrewAI • FastAPI • Next.js 16 • Cerebras LLM • "LLM reasons, code executes"

1. Project Overview

Axiom is an autonomous data-science platform. You give it a dataset (CSV) and an optional target column; it then runs a full ML lifecycle with no manual ML decisions required: profiling, cleaning, feature engineering, splitting, parallel model training, error auditing, hyperparameter improvement, and finalization with SHAP explainability and a markdown/PDF report.

Design principle — "LLM reasons, code executes." CrewAI agents direct the workflow through task descriptions and reasoning, but every piece of ML logic is deterministic Python inside tool/service classes. The LLM never computes metrics or mutates data directly — it decides *what* to do; the code decides *how*.

The system ships with two interchangeable entry points and a tiered web UI:

- **main.py** — a command-line interface that runs the full CrewAI pipeline.
- **api.py** — a FastAPI REST backend that powers the web app (upload, run, status, results, agents, visualizations, auth).
- **frontend/** — a Next.js 16 web app with a *Free* tier (one-click full pipeline) and an *Enterprise* tier (per-agent console, custom workflows, analytics, artifacts, experiments).

Tech Stack

Layer	Technologies
Orchestration	CrewAI, crewai-tools
Backend / API	FastAPI, Uvicorn, Pydantic v2, pydantic-settings
ML & Data	Pandas, NumPy, scikit-learn, XGBoost, LightGBM, Optuna
Explainability	SHAP, custom visualization engine (matplotlib)
Persistence	SQLAlchemy + SQLite (WAL), Joblib (model artifacts)
Auth	bcrypt / passlib, HTTP Bearer tokens
Export	openpyxl (Excel), reportlab (PDF reports)
LLM Provider	Cerebras via LiteLLM (bundled with CrewAI)
Frontend	Next.js 16 (Turbopack), React 19, TypeScript, Tailwind v4
FE Libraries	Zustand, @xyflow/react, Recharts, Framer Motion, lucide-react, react-syntax-highlighter, date-fns
Tooling / Tests	pytest, pytest-cov, pytest-mock, ESLint, psutil

2. System Architecture

```
main.py (CLI) → pipeline/orchestrator.py → CrewAI Crew (manager + 8 worker agents)
api.py (FastAPI REST) → core/agent_runner.py (direct, no-CrewAI runner)
frontend (Next.js) → /api/* proxied to http://127.0.0.1:8000
```

Two execution paths share the same agents and the same single source of truth (**PipelineState**):

- **CrewAI path** (CLI / Free full-pipeline) — the orchestrator assembles a Crew where the manager agent coordinates the 8 worker agents, with checkpointing and retry between stages.
- **Direct-runner path** (Enterprise / API) — `core/agent_runner.py` executes agents one at a time without CrewAI overhead, capturing per-agent timing, memory, metrics, logs, artifacts, and visualizations.

Repository Layout

Path	Purpose
<code>main.py</code>	CLI entry point (argparse): <code>--data</code> , <code>--target</code> , <code>--config</code> , <code>--run-id</code> , <code>--resume</code> , <code>--verbose</code>

Path	Purpose
api.py	FastAPI backend (~1,000 lines): all REST endpoints, auth, run tracking, exports
database.py	SQLAlchemy models + SQLite engine (User, UserSession, PromptHistory)
agents/	8 worker agents + 1 manager, each with its own service/tool module
core/	config, state, agent_runner, model_registry, metrics, validation, logging, utils, exceptions, constants
pipeline/	orchestrator.py (CrewAI assembly + retry) and checkpoint.py
visualization/	engine.py (chart engine), pdf_report.py (styled PDF), theme.py
configs/default.yaml	Base configuration for every pipeline stage
scripts/	generate_sample_data.py, run_pipeline.py
tests/	pytest suite (config, state, metrics, model_registry, validation, agents, checkpoint)
frontend/	Next.js 16 web application (src/app, components, store, lib)
artifacts/ reports/ logs/ data/	Run outputs, generated reports, structured logs, SQLite DB & datasets

3. Backend

The backend is the autonomous ML engine plus a FastAPI service layer. It has three faces: the **agents** that do the work, the **core/pipeline** machinery that orchestrates them, and the **FastAPI API** that exposes everything to the web app.

3.1 The Agents (8 workers + 1 manager)

Each agent lives in its own package under `agents/` and follows the same pattern: a CrewAI agent definition for reasoning, and a deterministic Python service that performs the actual ML work. The canonical agent IDs below must match the keys in `core/agent_runner.py` `AGENT_REGISTRY` and the frontend agent list.

Agent ID	Module	Responsibility
<code>data_collection</code>	<code>agents/data_collection/</code>	CSV/DB ingestion, dataset profiling, automatic problem-type detection (classification / regression / clustering)
<code>preprocessing</code>	<code>agents/preprocessing/</code>	Cleaning, imputation, outlier handling (IQR), duplicate removal, data-quality scoring
<code>feature_engineering</code>	<code>agents/feature_engineering/</code>	Encoding, scaling, variance/correlation filtering, SelectKBest, datetime feature extraction
<code>data_splitting</code>	<code>agents/splitting/</code>	Stratified / standard train-val-test splits with leakage prevention
<code>model_training</code>	<code>agents/training/</code>	Parallel multi-model training, model registry, cross-validated metric evaluation
<code>error_detection</code>	<code>agents/error_detection/</code>	Audits for overfitting, leakage, class imbalance, and underperformance vs. thresholds
<code>improvement</code>	<code>agents/improvement/</code>	Hyperparameter tuning via RandomizedSearchCV or Optuna; drives the retry loop
<code>finalization</code>	<code>agents/finalization/</code>	Saves artifacts, computes SHAP explainability, generates the markdown/PDF report
<code>manager</code>	<code>agents/manager/</code>	CrewAI manager agent that coordinates the worker crew (CrewAI path only)

3.2 Core Modules (core/)

Module	What it provides
<code>config.py</code>	Pydantic v2 Settings. Layered config: YAML → environment variables → programmatic overrides. <code>load_settings()</code> is the entry.
<code>state.py</code>	<code>PipelineState</code> — the Pydantic model that is the single source of truth passed across all agents.
<code>agent_runner.py</code>	<code>AGENT_REGISTRY</code> + direct agent execution (no CrewAI). Captures duration, memory (psutil), metrics, logs, artifacts, visualizations per agent.
<code>model_registry.py</code>	Unified registry of models with default params and hyperparameter search spaces.
<code>metrics.py</code>	Metric computation for classification / regression / clustering.
<code>validation.py</code>	Input and data validation helpers.
<code>logging_config.py</code>	Structured JSON logging with rotation (<code>setup_logging</code> , <code>get_logger</code>).
<code>utils.py</code>	<code>generate_run_id</code> , timestamps, directory helpers.
<code>constants.py</code> / <code>exceptions.py</code>	Shared constants and the custom exception hierarchy.

3.3 Pipeline Modules (pipeline/)

- **orchestrator.py** — PipelineOrchestrator assembles the CrewAI Crew and runs the full pipeline with checkpoint support and a retry loop driven by the improvement agent.
- **checkpoint.py** — saves/loads the PipelineState as JSON after each stage, enabling --resume.

3.4 Database Layer (database.py)

SQLAlchemy 2.0 over SQLite, stored at data/axiom.db. WAL journal mode and foreign keys are enabled on connect. Tables auto-create on import. Three models:

Model / Table	Key fields
User → users	id, email (unique), username, password_hash (bcrypt), created_at, last_login
UserSession → sessions	id, user_id (FK, cascade), token (64-char hex), created_at, expires_at; is_expired property
PromptHistory → prompt_history	id, user_id (FK), run_id, data_path, target_column, mode (free enterprise), status, result_summary (JSON), created_at

3.5 FastAPI Application (api.py)

App title "Axiom — Autonomous AI Platform", version 2.0.0. CORS is fully open. Authentication uses HTTP Bearer tokens validated against the sessions table (get_current_user dependency). Running pipelines are tracked in an in-memory `_active_runs` dict and executed on background threads, so the API can poll status without blocking. Served by Uvicorn on `127.0.0.1:8000`.

Authentication endpoints

Method & Path	Description
POST /api/auth/signup	Create account (email, username, password). Rejects duplicate email; hashes password with bcrypt.
POST /api/auth/login	Verify credentials, issue a session token, update last_login.
GET /api/auth/me	Return the current authenticated user.
POST /api/auth/logout	Invalidate the caller's session token.
GET /api/auth/history	Return the user's PromptHistory (past runs).

Pipeline & run endpoints

Method & Path	Description
GET /	Serves a landing HTML page.
POST /api/upload	Upload a CSV; returns columns, dtypes, row count, a preview, and auto-generated visualizations.
POST /api/init-run	Initialize a run context (data_path, target) without starting work — used by the Enterprise Agent Console.
POST /api/run	Start the full pipeline (Free mode) on a background thread; returns a run_id.
POST /api/workflow/run	Enterprise: run a custom ordered list of agents on a dataset.
POST /api/agent/{agent_name}/run	Enterprise: run a single agent against an existing run context, with optional config.
GET /api/agent/{agent_name}/output/{run_id}	Fetch one agent's full output (summary, metrics, logs, artifacts, visualizations).
GET /api/status/{run_id}	Poll run status: starting running completed failed, current_stage, completed_stages, logs, agent_outputs.
GET /api/results/{run_id}	Full results: problem type, best model/metric, dataset & preprocessing stats, features, models, errors, experiment history.
GET /api/runs	List all runs (experiment tracking) with status, timing, best model, stage counts.

Reporting, explainability & export endpoints

Method & Path	Description
GET /api/report/{run_id}	Return the generated markdown report text.
GET /api/report/{run_id}/pdf	Download the styled PDF report (reportlab via visualization/pdf_report.py).
GET /api/shap/{run_id}	Return SHAP feature-importance values for the best model.
GET /api/visualizations/{run_id}	List run visualizations; optional ?category=basic advanced filter.
POST /api/visualizations/{run_id}/generate	Generate a specific visualization on demand (type + params).
GET /api/export/{run_id}/excel	Download all agent outputs as a multi-sheet Excel workbook (openpyxl).

All non-auth data endpoints flow through the same PipelineState, so the Free full-pipeline and Enterprise per-agent paths return identically shaped agent outputs.

3.6 LLM Provider

Reasoning runs on **Cerebras** through LiteLLM (bundled with CrewAI) — no extra package needed. Default model `cerebras/llama3.1-8b`, temperature 0.1, max_tokens 4096. Set `CEREBRAS_API_KEY` in `.env`.

4. Frontend

A Next.js 16 (App Router, Turbopack) application written in TypeScript with Tailwind v4. **Important:** this Next.js version has breaking changes vs. older releases — per `frontend/AGENTS.md`, consult `node_modules/next/dist/docs/` before changing framework code. The dev server proxies `/api/*` to the FastAPI backend, so the browser only ever talks to one origin.

4.1 Two product tiers

- **Free tier** — upload a dataset, pick a target, click run; watch the full pipeline progress and view results/report. One-click autonomous ML.
- **Enterprise tier** — an agent console plus dedicated views for workflows, agents, analytics, artifacts, and experiments; run agents individually or as custom workflows.

4.2 Routes (`src/app/`)

Route	File	Purpose
<code>/</code>	<code>app/page.tsx</code>	Public landing page
<code>/auth</code>	<code>app/auth/page.tsx</code>	Login / signup
<code>/free</code>	<code>app/free/page.tsx</code>	Free-tier upload & launch
<code>/free/pipeline/[runId]</code>	<code>app/free/pipeline/[runId]/page.tsx</code>	Live pipeline progress (Free)
<code>/free/results/[runId]</code>	<code>app/free/results/[runId]/page.tsx</code>	Results & report (Free)
<code>/pipeline/[runId]</code>	<code>app/pipeline/[runId]/page.tsx</code>	Generic pipeline progress view
<code>/report/[runId]</code>	<code>app/report/[runId]/page.tsx</code>	Generic report view
<code>/enterprise</code>	<code>app/enterprise/page.tsx</code>	Enterprise dashboard / agent console
<code>/enterprise/workflow</code>	<code>app/enterprise/workflow/page.tsx</code>	Build & run custom agent workflows
<code>/enterprise/agents</code>	<code>app/enterprise/agents/page.tsx</code>	Per-agent inspection & execution
<code>/enterprise/analytics</code>	<code>app/enterprise/analytics/page.tsx</code>	Metrics & charts
<code>/enterprise/artifacts</code>	<code>app/enterprise/artifacts/page.tsx</code>	Models, files, downloads
<code>/enterprise/experiments</code>	<code>app/enterprise/experiments/page.tsx</code>	Run history / experiment tracking

4.3 Components, state & libraries

Layout components (`src/components/layout/`)

- **AppShell.tsx** — top-level layout wrapper.
- **Navbar.tsx** — top navigation bar.
- **Sidebar.tsx** — collapsible side navigation.
- **ModeSwitch.tsx** — toggle between Free and Enterprise modes.

UI components (`src/components/ui/`)

- **AnimatedCounter.tsx** — animated numeric counter for stats.

State management — Zustand (`src/store/`)

Store	Responsibility
<code>useAuthStore</code>	user, token, isAuthenticated; <code>setAuth()</code> / <code>logout()</code> . Persisted; injects the Bearer token into API calls.
<code>useAppStore</code>	mode (free enterprise), current dataset preview, activeRunId, experiments list (capped at 100), sidebar state. Persisted.

Store	Responsibility
useAgentStore	Per-agent status, outputs, logs, and visualizations for a run; selectors like <code>getRunningAgent()</code> , <code>isAllCompleted()</code> .

Library layer (src/lib/)

- **api.ts** — the typed API client. Exposes every backend call (`auth`, `upload`, `startPipeline`, `initRun`, `runSingleAgent`, `runWorkflow`, `getStatus`, `getResults`, `getReport`, `downloadReportPdf`, `getShapData`, `getAgentOutput`, `getVisualizations`, `generateVisualization`, `listExperiments`, `exportRunExcel`) plus shared response interfaces and an `authFetch()` wrapper that attaches the token and normalizes errors.
- **types.ts** — shared TypeScript domain types (`AgentId`, `AgentStatus`, `Experiment`, `DatasetPreview`, etc.).
- **animations.ts** — Framer Motion animation presets.

4.4 Backend proxy (next.config.ts)

Turbopack root is pinned to the frontend folder (so a stray lockfile higher up the tree can't mis-set the workspace root). A rewrite forwards every request:

```
/api/:path* → http://127.0.0.1:8000/api/:path*
```

This means the frontend code calls relative `/api/...` URLs and the dev server transparently proxies them to FastAPI — no CORS juggling, no hardcoded backend host in the client.

5. End-to-End Data Flow

Free tier (full autonomous pipeline)

- 1 User signs up / logs in → `useAuthStore` stores the Bearer token.
- 2 User uploads CSV → `POST /api/upload` returns preview + auto-visualizations.
- 3 User picks a target and clicks Run → `POST /api/run` starts a background thread, returns `run_id`.
- 4 Pipeline page polls `GET /api/status/{run_id}` for stage progress and per-agent outputs.
- 5 On completion → `GET /api/results/{run_id}`, `/report`, `/shap`, `/report/{id}/pdf`, `/export/{id}/excel`.

Enterprise tier (granular control)

- 1 `POST /api/init-run` establishes a run context without executing anything.
- 2 Run agents individually via `POST /api/agent/{name}/run`, or as an ordered set via `POST /api/workflow/run`.
- 3 Inspect each agent with `GET /api/agent/{name}/output/{run_id}`; track all runs via `GET /api/runs`.
- 4 Generate or filter visualizations on demand; export artifacts and reports.

6. Configuration

6.1 Environment variables (.env)

Variable	Default / Example	Meaning
CEREBRAS_API_KEY	csk-...	Required — Cerebras key for agent reasoning
LLM_MODEL	cerebras/llama3.1-8b	LiteLLM-format model identifier
LLM_TEMPERATURE	0.1	Sampling temperature
LLM_MAX_TOKENS	4096	Max tokens per completion
LOG_LEVEL	INFO	DEBUG INFO WARNING ERROR CRITICAL
RANDOM_SEED	42	Reproducibility seed (optional override)
MAX_RETRIES	3	Improvement-loop retry cap (optional)
ARTIFACT_DIR / REPORT_DIR / LOG_DIR	artifacts / reports / logs	Output path overrides (optional)
DATABASE_URL	sqlite:///data/source.db	Optional — for SQL data ingestion
MLFLOW_TRACKING_URI / _EXPERIMENT_NAME	(unset)	Optional MLflow tracking

6.2 Pipeline config (configs/default.yaml)

Every stage is tunable from YAML; any value can be overridden by an environment variable prefixed with its section name (e.g. `PIPELINE_RANDOM_SEED=123`). Key defaults:

Section	Notable settings
pipeline	<code>random_seed=42</code> , <code>max_retries=3</code> , <code>checkpoint_enabled=true</code> , <code>artifact/report/data/log</code> dirs
data_collection	<code>chunk_size=50000</code> , <code>max_columns=500</code> , <code>sample_rows_for_profiling=10000</code>
preprocessing	<code>outlier_method=iqr</code> , <code>iqr_multiplier=1.5</code> , <code>max_null_threshold=0.70</code> , <code>max_cardinality=50</code>
feature_engineering	<code>max_onehot_cardinality=15</code> , <code>variance_threshold=0.01</code> , <code>correlation_threshold=0.95</code> , <code>select_k_best=20</code> , <code>scaling=standard</code>

Section	Notable settings
splitting	train=0.70, val=0.15, test=0.15
training	n_jobs=-1, cv_folds=5, timeout_per_model=300s
error_detection	min_classification_f1=0.30, min_regression_r2=0.10, overfitting_threshold=0.15, max_feature_count=500
improvement	tuning_iterations=50, early_stopping_rounds=10, use_optuna=false
logging	level=INFO, max_bytes=10MB, backup_count=5, json_format=true

7. Running the Project

7.1 One command (recommended)

From the `frontend/` directory, a single script launches both the FastAPI backend and the Next.js dev server together (via `dev-all.mjs`). It prefers the project's venv Python, pipes both logs with `[api]/[web]` prefixes, and tears both down if either exits:

```
cd frontend
npm install # first time only
npm run dev
```

- Backend (FastAPI / Uvicorn) → `http://127.0.0.1:8000`
- Frontend (Next.js) → `http://localhost:3000` (open this)

7.2 Run the halves separately

Command (in frontend/)	Starts
<code>npm run dev:web</code>	Next.js dev server only (next dev)
<code>npm run dev:api</code>	FastAPI backend only (python ../api.py)
<code>npm run build / npm run start</code>	Production build / serve
<code>npm run lint</code>	ESLint

Or start the backend directly from the project root:

```
python api.py # uvicorn on 127.0.0.1:8000
```

7.3 Run the pipeline from the CLI (no web UI)

```
python main.py --data data/sample_iris.csv --target species
python main.py --data data/housing.csv --target price --config configs/default.yaml
python main.py --data data/x.csv --target y --resume # resume from checkpoint
```

Omit `--target` to run in clustering mode. Use `--verbose` for `DEBUG` logging and `--run-id` to set a custom run identifier.

7.4 First-time setup

- 1 Create & activate a Python 3.11+ virtualenv (the repo ships a venv/).
- 2 `pip install -r requirements.txt`
- 3 Copy `.env.example` to `.env` and set `CEREBRAS_API_KEY`.
- 4 `cd frontend && npm install`
- 5 `npm run dev` → open `http://localhost:3000`

7.5 Tests

```
pytest # full suite
pytest --cov # with coverage
```

8. Outputs & Artifacts

Location	Contents
<code>artifacts/<run_id>/</code>	Trained models (Joblib), metrics, checkpoints, SHAP values
<code>reports/<run_id>/</code>	Markdown report (and PDF when exported)
<code>logs/</code>	Rotating structured JSON logs
<code>data/axiom.db</code>	SQLite database (users, sessions, prompt history)

Axiom — Autonomous Data Scientist Platform. This document was generated as a complete reference to the frontend and backend. Endpoint names, routes, agents, config keys, and run commands reflect the current codebase.